What a BNF Grammar Defines

A BNF grammar is a *recipe* for building strings.

- Start at the **start symbol** (the first **non-terminal** rule)
- Repeatedly replace a **non-terminal** using a rule (a **production**)
- Stop when you have only **terminals**

The **language** of the grammar is the set of *all* strings you can build this way.

BNF Grammars

Sam Ogden

Motivating Example

Writing a JSON Parser

```
{
  "1": {
    "location": "assign-
ostep7",
    "weight": 20
  },
  "2": {
    "location": "assign-
ostep8",
    "weight": 20
  },
  "3": {
    "location": "assign-msh3",
    "weight": 60
```

(example input text)

Someone asks you to build a JSON parser. How do you approach it?

Input: A blob of text
Output: A dictionary ***or*** error

What is a “language”?

A language is a set of strings

- `{"a", "ab", "abc"}`
- `{"x=1", "x=y"}`

Infinite languages can’t be defined by listing all their strings

- `{"a", "ab", ...}`

But we have to be careful because these aren’t *explicit* definitions

Is the next string `"aba"` or `"abc"`?

Backus-Naur Form (BNF) Grammars

```
1 R_1 ::= a | a R_2
2 R_2 ::= b | b R_1
```

Three parts of grammars:

1. **non-terminal** *symbols* are placeholders that *must* be replaced
2. **terminals** are concrete parts of the output string
3. **productions** are what we replace **non-terminals** with
   - e.g. "**b**$\mathbf{R_1}$" in the above example

Using BNFs

BNF Practice

Let’s try generating some strings

```
R_1 ::= foo | bar
```

Let’s try generating some strings

```
1 R_1 ::= [ R_2 ]
2 R_2 ::= a | b | c
```

Let’s try generating some strings

```
R_1 ::= a R_1 b
```

Let’s try *parsing* some strings

```
1 R_1 ::= a R_1 b
```

aabb

Let’s try *parsing* some strings

```
1 R_1 ::= a R_1 b
```

abb

Let’s try *parsing* some strings

```
1 R_1 ::= R_1 + R_1
```

abb

Let’s try *parsing* some strings

```
R_1 ::= a R_1 b | 1 | x
```

$$1 + x + x$$

Let’s try *parsing* some strings

```
1 R_1 ::= a R_1 b | 1 | x
```

$$1 + x +$$